

Normalization Methods

Hyoungh Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Main Ideas	2
2.1	Data Normalization	2
2.2	Batch Normalization	2
2.2.1	Inference	2
2.3	Layer Normalization	3

1 Introduction

We introduce methods of **normalization**, which helps deep neural network to be trained. Three methods will be discussed: **Data normalization**, **layer normalization** and **batch normalization**.

2 Main Ideas

2.1 Data Normalization

Data normalization is to normalize the test data along mini-batch dataset. Since each feature values are distributed along different ranges, some feature may be ignored, or overemphasized. Normalizing the data solves this problem, by making all the feature values to have zero mean and unit variance.

My experience about the effect of data normalization: Polynomial regression exploded at degree 7 without data normalization. But with data normalization, it did not explode even when I used degree 500 for polynomial features.

2.2 Batch Normalization

Batch normalization is to normalize the hidden layers about the batch dataset. We first forward pass the mini-batch data, and then, we calculate the mean and standard deviation of the hidden units calculated about each data in the mini-batch dataset. We can apply this at pre-activation or activated layer. In practice, either method can be used.

Let a_{ni} be the i -th pre-activation unit calculated with n -th element of mini-batch dataset. Then,

$$\mu_i = \frac{1}{K} \sum_{n=1}^K a_{ni} \quad (1)$$

$$\sigma_i^2 = \frac{1}{K} \sum_{n=1}^K (a_{ni} - \mu_i)^2 \quad (2)$$

After that, we can calculate the normalized units:

$$\hat{a}_{ni} = \frac{a_{ni} - \mu_i}{\sigma_i + \epsilon}$$

The normalized unit loses some of the degree of freedom. To compensate this, we can scale it by using two learnable parameters γ_i and β_i :

$$\tilde{a}_{ni} = \gamma_i \hat{a}_{ni} + \beta_i$$

2.2.1 Inference

In the inference step, we cannot use batch normalization, since there only a single data is used. To solve this, we can evaluate the mean and stdev across the whole hidden units. However, this is usually expensive, so we use **moving averages** calculated during the training process:

$$\bar{\mu}_i^{(\tau)} = \alpha \bar{\mu}_i^{(\tau-1)} + (1 - \alpha) \mu_i \quad (3)$$

$$\bar{\sigma}_i^{(\tau)} = \alpha \bar{\sigma}_i^{(\tau-1)} + (1 - \alpha) \sigma_i \quad (4)$$

calculated with τ -th mini-batch dataset, where $0 \leq \alpha \leq 1$.

2.3 Layer Normalization

Using batch normalization injects too much noise if the batch size is too small. Even when the dataset is too large, the data can be distributed to different GPUs, and batch normalization cannot be applied to the layer. **Layer normalization** is the alternative of batch normalization, which also uses training datasets to normalize the weights and biases.

Layer normalization normalize the hidden units with respect to the layer itself. Let's assume that we have hidden layer with M hidden units. Then, we calculate the mean and variance of hidden units;

$$\mu_n = \frac{1}{M} \sum_{i=1}^M a_{ni} \quad (5)$$

$$\sigma_n^2 = \frac{1}{M} \sum_{i=1}^M (a_{ni} - \mu_n)^2 \quad (6)$$

Then, as we did in batch normalization, we normalize the hidden units;

$$\hat{a}_{ni} = \frac{a_{ni} - \mu_n}{\sigma_n + \epsilon}$$

Note that the layer normalization can be applied during inference, since we can apply layer normalization to one data.